FRD

Functional Requirements Document (FRD)

AWS S3-Based Data Processing Pipeline

•

FR-INGEST-001

- Title: Ingest Customer CSV from S3

- Description:

The system shall read customer data in CSV format from the designated S3 input path.

- Preconditions:
- Customer CSV file exists at `s3://adif-sdlc/sdlc_wizard/customerdata/`
- File contains columns: CustId, Name, EmailId, Region

- Main Flow / Functional Steps:
- Locate CSV file in the specified S3 path
- Parse CSV with schema: CustId, Name, EmailId, Region
- Load data into memory for downstream processing

- Postconditions:
- Customer dataset is available for cleaning and transformation

- Assumptions:
- CSV file is well-formed and uses standard delimiters
- File encoding is UTF-8 or compatible

•

FR-INGEST-002

- Title: Ingest Order CSV from S3

- Description:

The system shall read order data in CSV format from the designated S3 input path.

- Preconditions:
- Order CSV file exists at `s3://adif-sdlc/sdlc_wizard/orderdata/`
- File contains columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Main Flow / Functional Steps:
- Locate CSV file in the specified S3 path
- Parse CSV with schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Load data into memory for downstream processing

- Postconditions:
- Order dataset is available for cleaning and transformation

- Assumptions:
- CSV file is well-formed and uses standard delimiters
- File encoding is UTF-8 or compatible

-

FR-CLEAN-001

- Title: Remove Null Values from Customer Dataset

- Description:

The system shall remove rows containing "Null" (string) or NULL values from the customer dataset.

- Preconditions:
- Customer dataset has been ingested

- Main Flow / Functional Steps:
- Scan all rows in customer dataset
- Identify rows where any column contains "Null" or NULL
- Remove identified rows from the dataset

- Postconditions:
- Customer dataset contains no "Null" or NULL values

- Assumptions:
- "Null" is case-sensitive or case-insensitive as per implementation
- Empty strings are not treated as NULL unless specified

-

FR-CLEAN-002

- Title: Remove Null Values from Order Dataset

- Description:

The system shall remove rows containing "Null" (string) or NULL values from the order dataset.

- Preconditions:
- Order dataset has been ingested

- Main Flow / Functional Steps:
- Scan all rows in order dataset
- Identify rows where any column contains "Null" or NULL
- Remove identified rows from the dataset

- Postconditions:
- Order dataset contains no "Null" or NULL values

- Assumptions:
- "Null" is case-sensitive or case-insensitive as per implementation
- Empty strings are not treated as NULL unless specified

-

FR-CLEAN-003

- Title: Remove Duplicates from Customer Dataset

- Description:

The system shall remove duplicate rows from the customer dataset.

- Preconditions:
- Customer dataset has been cleaned of null values

- Main Flow / Functional Steps:
- Identify duplicate rows based on all columns
- Retain one instance of each unique row
- Remove all other duplicate instances

- Postconditions:
- Customer dataset contains only unique rows

- Assumptions:
- Duplicates are determined by exact match across all columns
- No specific ordering preference for which duplicate to retain

-

FR-CLEAN-004

- Title: Remove Duplicates from Order Dataset

- Description:

The system shall remove duplicate rows from the order dataset.

- Preconditions:
- Order dataset has been cleaned of null values

- Main Flow / Functional Steps:
- Identify duplicate rows based on all columns
- Retain one instance of each unique row
- Remove all other duplicate instances

- Postconditions:
- Order dataset contains only unique rows

- Assumptions:
- Duplicates are determined by exact match across all columns
- No specific ordering preference for which duplicate to retain

-

FR-CATALOG-001

- Title: Register Customer Table in Glue Catalog

- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog.

- Preconditions:
- Customer dataset has been cleaned
- Glue database `gen_ai_poc_databrickscoe` exists

- Main Flow / Functional Steps:
- Create or update table `sdlc_wizard_customer` in database `gen_ai_poc_databrickscoe`
- Define schema: CustId, Name, EmailId, Region
- Register table metadata in Glue Catalog

- Postconditions:
- Table `sdlc_wizard_customer` is queryable via Athena and accessible by Glue jobs

- Assumptions:
- Appropriate IAM permissions exist for Glue Catalog operations
- Table location is managed by Glue or explicitly defined

-

FR-CATALOG-002

- Title: Register Order Table in Glue Catalog

- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog.

- Preconditions:
- Order dataset has been cleaned
- Glue database `gen_ai_poc_databrickscoe` exists

- Main Flow / Functional Steps:
- Create or update table `sdlc_wizard_order` in database `gen_ai_poc_databrickscoe`
- Define schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Register table metadata in Glue Catalog

- Postconditions:
- Table `sdlc_wizard_order` is queryable via Athena and accessible by Glue jobs

- Assumptions:
- Appropriate IAM permissions exist for Glue Catalog operations
- Table location is managed by Glue or explicitly defined

-

FR-SCD-001

- Title: Implement SCD Type 2 for Order Summary Table

- Description:

The system shall maintain a Slowly Changing Dimension Type 2 table (`ordersummary`) using Apache Hudi on S3.

- Preconditions:
- Customer and order tables are available in Glue Catalog
- Target S3 location `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/` is accessible

- Main Flow / Functional Steps:
- Join customer and order datasets on CustId
- Detect changes in joined records
- Insert new rows with IsActive = true, current StartDate, null EndDate, and OpTs timestamp
- Update previous rows by setting IsActive = false, populating EndDate, and updating OpTs
- Write results to S3 using Hudi format

- Postconditions:
- `ordersummary` table reflects current and historical states
- All active records have IsActive = true
- Closed records have IsActive = false with valid EndDate

- Assumptions:
- Change detection is based on all non-metadata columns
- OpTs is system-generated timestamp
- Hudi configuration supports upsert operations

-

FR-SCD-002

- Title: Register Order Summary Table in Glue Catalog

- Description:

The system shall register the `ordersummary` table in the AWS Glue Data Catalog.

- Preconditions:
- `ordersummary` data has been written to S3 in Hudi format
- Glue database `gen_ai_poc_databrickscoe` exists

- Main Flow / Functional Steps:
- Create or update table `ordersummary` in database `gen_ai_poc_databrickscoe`
- Define schema including IsActive, StartDate, EndDate, OpTs
- Register Hudi table metadata in Glue Catalog

- Postconditions:
- Table `ordersummary` is queryable via Athena with SCD Type 2 history

- Assumptions:
- Glue Catalog supports Hudi table format
- Appropriate IAM permissions exist

•

FR-INCR-001

- Title: Trigger Incremental SCD2 Updates on Customer Changes

- Description:

The system shall detect customer data updates and trigger incremental SCD Type 2 updates in the `ordersummary` table.

- Preconditions:
- Customer dataset has new or updated records
- `ordersummary` table exists

- Main Flow / Functional Steps:
- Identify changed or new customer records
- Re-join affected customers with order data
- Apply SCD Type 2 logic to update `ordersummary`
- Maintain IsActive, StartDate, EndDate, OpTs for affected rows

- Postconditions:
- `ordersummary` reflects latest customer changes incrementally
- Historical records remain intact

- Assumptions:
- Change detection mechanism is in place (e.g., timestamp, hash comparison)
- Only affected records are processed for efficiency

•

FR-AGG-001

- Title: Generate Customer Aggregate Spend Table

- Description:

The system shall create an aggregation table (`customeraggregatespend`) summarizing total spending per customer.

- Preconditions:
- `ordersummary` table is available
- Target S3 location `s3://adif-sdlc/analytics/customeraggregatespend/` is accessible

- Main Flow / Functional Steps:

- Read active records from `ordersummary` (IsActive = true)
- Calculate TotalAmount per customer (sum of PricePerUnit Qty)
- Group by customer Name and Date
- Write results to S3 with columns: Name, TotalAmount, Date

- Postconditions:
- `customeraggregatespend` table contains aggregated spending data

- Assumptions:
- TotalAmount calculation uses PricePerUnit and Qty from order data
- Date represents the aggregation period or transaction date

-

FR-AGG-002

- Title: Register Customer Aggregate Spend Table in Glue Catalog

- Description:

The system shall register the `customeraggregatespend` table in the AWS Glue Data Catalog.

- Preconditions:
- `customeraggregatespend` data has been written to S3
- Glue database `gen_ai_poc_databrickscoe` exists

- Main Flow / Functional Steps:
- Create or update table `customeraggregatespend` in database `gen_ai_poc_databrickscoe`
- Define schema: Name, TotalAmount, Date
- Register table metadata in Glue Catalog

- Postconditions:
- Table `customeraggregatespend` is queryable via Athena

- Assumptions:
- Appropriate IAM permissions exist
- Table format is compatible with Athena queries

-

FR-ARCH-001

- Title: Use AWS Glue for ETL Processing

- Description:

The system shall execute all ETL operations (ingestion, cleaning, transformation, SCD, aggregation) using AWS Glue with Spark and Hudi.

- Preconditions:
- AWS Glue environment is configured
- Spark and Hudi libraries are available

- Main Flow / Functional Steps:
- Define Glue jobs for each ETL stage
- Execute jobs using Spark engine
- Leverage Hudi for SCD Type 2 operations

- Postconditions:
- All data processing is performed within AWS Glue

- Assumptions:
- Glue job configurations include necessary dependencies
- Sufficient Glue DPU capacity is allocated

-

FR-ARCH-002

- Title: Store All Data on AWS S3

- Description:

The system shall use AWS S3 as the storage layer for all input, intermediate, curated, and analytics data.

- Preconditions:
- S3 buckets and paths are provisioned
- IAM roles grant read/write access

- Main Flow / Functional Steps:
- Read source data from S3 input paths
- Write processed data to S3 curated and analytics paths
- Maintain data in appropriate formats (CSV, Hudi, Parquet)

- Postconditions:
- All data resides in S3 with appropriate lifecycle policies

- Assumptions:

- S3 bucket versioning and encryption are configured as per security policies
- Data retention policies are defined

-

FR-ARCH-003

- Title: Enable Querying via Amazon Athena

- Description:

The system shall enable SQL-based querying of all registered tables using Amazon Athena.

- Preconditions:
- Tables are registered in Glue Data Catalog
- Athena workgroup is configured

- Main Flow / Functional Steps:
- Users execute SQL queries via Athena console or API
- Athena reads table metadata from Glue Catalog
- Athena scans data from S3 locations

- Postconditions:
- All tables are queryable via standard SQL

- Assumptions:
- Athena query results are stored in a designated S3 bucket
- Users have appropriate IAM permissions

-

FR-ARCH-004

- Title: Monitor ETL Jobs with CloudWatch

- Description:

The system shall log and monitor all Glue job executions using Amazon CloudWatch.

- Preconditions:
- CloudWatch logging is enabled for Glue jobs
- CloudWatch alarms are configured (optional)

- Main Flow / Functional Steps:
- Glue jobs emit logs to CloudWatch Logs

- Metrics (duration, success/failure, DPU usage) are tracked
- Alerts are triggered on job failures or performance thresholds

- Postconditions:
- ETL job health and performance are observable via CloudWatch

- Assumptions:
- Log retention policies are defined
- Appropriate IAM roles grant CloudWatch access

-

FR-ARCH-005

- Title: Optional Governance with AWS Lake Formation

- Description:

The system may optionally use AWS Lake Formation for data governance, access control, and auditing.

- Preconditions:
- Lake Formation is enabled in the AWS account
- Data catalog is registered with Lake Formation

- Main Flow / Functional Steps:
- Define Lake Formation permissions for databases and tables
- Enforce fine-grained access control
- Audit data access via Lake Formation logs

- Postconditions:
- Data access is governed and auditable

- Assumptions:
- Lake Formation policies are defined by data governance team
- Integration with Glue Catalog is configured

-

Global Assumptions

- All AWS services are deployed in the same region for optimal performance
- IAM roles and policies follow least-privilege principles
- Data formats (CSV, Hudi, Parquet) are compatible with Glue and Athena
- Network connectivity and VPC configurations allow service communication

- No real-time streaming requirements; batch processing is acceptable
- Date fields use ISO 8601 or consistent format across datasets
- CustId is the primary key for customer data and foreign key in order data
- OrderId is the primary key for order data
- TotalAmount calculation does not include taxes or discounts unless specified
- SCD Type 2 logic assumes no backdated changes; all updates are forward-dated